

Challenge Write-Up Template

Challenge Information

Student Name	Pascua, Diana KC S.
Section / Class	BSIT-3G
Challenge Name	My Git
Category	General Skills
Points / Difficulty	Easy
Date Solved	8/14/26
Flag Format Given	picoCTF{....}

1. Challenge Description

This challenge, called "MyGit," gave me SSH access to a remote Git repository (foggy-cliff.picoctf.net) through a webshell. Cloning the repo showed a single README.md file explaining the rules: the server has a hook watching for a file named flag.txt, and it will only fill that file in with the real flag if the commit that adds it is authored by "root" with the email "root@picoctf." My job was to figure out how Git tracks authorship and use that to convince the server I was root, even though I was just logged in as a normal low-privilege user (PascuaDiana-academy)

2. Initial Analysis / Reconnaissance

The first thing I did after cloning was list the repo's contents with ls -la and then cat the README.md. That's a habit I try to keep for every challenge — read whatever documentation the challenge gives you before touching anything else, because it usually tells you the actual win condition instead of making you guess. The README made it clear this wasn't a cryptography or web challenge; it was purely about Git internals, specifically how Git decides who the "author" of a commit is. That told me this was a general-skills / Git-trust challenge: Git stores author name and email as plain metadata supplied by the client (via git config), and never verifies that the person running the command is actually who they claim to be. The whole challenge is built around exploiting that trust gap.

3. Tools and Commands Used

Tool / Command	What it was used for	Result / Output
<code>git clone ssh://git@foggy-cliff.picoc.tf.net:54145/git/challenge.git</code>	Clone the challenge's remote Git repo to my local webshell so I could look at it.	<i>Accepted the host key, entered the given password, and got a local copy in ~/challenge with a .git folder and a README.md</i>
<code>cat README.md</code>	Read the repo's instructions to see what the challenge actually wanted.	README said the flag only gets written into flag.txt if that file is pushed by a commit authored as "root:root@picoc.tf". Told me exactly what identity to fake.
<code>git config user.name "root" git config user.email "root@picoc.tf"</code>	Set the local repo's commit author identity, since Git trusts whatever name/email you configure — it does no verification.	No output, but every commit I make from here on is now authored as root <root@picoc.tf>.
<code>echo "placeholder" > flag.txt</code>	Create the flag.txt file the README asked for, since it didn't exist yet.	A new flag.txt containing the word 'placeholder' — the real content didn't matter, only the filename and the commit author.
<code>git add flag.txt git commit -m "add flag.txt"</code>	Stage and commit the new file under the spoofed root identity.	[master 04f1c7b] add flag.txt — 1 file changed, 1 insertion(+), create mode 100644 flag.txt
<code>git push</code>	Send the commit back to the server so its hook could inspect the author field.	remote: Author matched and flag.txt found in commit... remote: Congratulations! You have

		successfully impersonated the root user remote: Here's the flag: picoCTF{....}
--	--	--

4. Solution Walkthrough

Step 1

Clone the repository and read the rules

I ran `git clone ssh://git@foggy-cliff.picoc.tf.net:54145/git/challenge.git`, accepted the new host key, and authenticated with the provided password. Inside the challenge folder, `cat README.md` spelled out the win condition directly: "Only `flag.txt` pushed by `root:root@picoc.tf` will be updated with the flag." That single line was really the whole puzzle — I didn't need real root access to the server, I just needed a commit whose author metadata said `root:root@picoc.tf`.

```
PascuaDiana-academy@webshell:~$ git clone ssh://git@foggy-cliff.picoc.tf.net:54145/git/challenge.git
Cloning into 'challenge'...
The authenticity of host '[foggy-cliff.picoc.tf.net]:54145 ([3.15.249.208]:54145)' can't be established.
ED25519 key fingerprint is SHA256:Grw7IvZgdCDbXv3DtQ70/6WkHA2q3XhT+sfva8nLT38.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[foggy-cliff.picoc.tf.net]:54145' (ED25519) to the list of known hosts.
git@foggy-cliff.picoc.tf.net's password:
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0)
Receiving objects: 100% (3/3), done.
PascuaDiana-academy@webshell:~$ cd challenge
PascuaDiana-academy@webshell:~/challenge$ ls -la
total 4
drwxrwxr-x 3 PascuaDiana-academy PascuaDiana-academy 35 Aug 14 09:56 .
drwxr-xr-x 4 PascuaDiana-academy PascuaDiana-academy 104 Aug 14 09:56 ..
drwxrwxr-x 8 PascuaDiana-academy PascuaDiana-academy 163 Aug 14 09:56 .git
-rw-rw-r-- 1 PascuaDiana-academy PascuaDiana-academy 155 Aug 14 09:56 README.md
PascuaDiana-academy@webshell:~/challenge$ cat README.md
# MyGit

### If you want the flag, make sure to push the flag!

Only flag.txt pushed by ``root:root@picoc.tf`` will be updated with the flag.

GOOD LUCK!
PascuaDiana-academy@webshell:~/challenge$ git config user.name "root"
```

Step 2

Impersonate root's Git identity

Git identity is entirely self-reported. Anyone can set `git config user.name` and `git config user.email` to whatever they want, and Git will happily stamp that onto every commit they make afterward. So I ran: `git config user.name "root"` `git config user.email "root@picoc.tf"` This didn't give me any real system privileges — it just changed how my next commits would be labeled.

Step 3

Create and commit `flag.txt` under that identity

I created the file the hook was looking for with `echo "placeholder" > flag.txt`, then staged and committed it with `git add flag.txt` and `git commit -m "add flag.txt"`. The content of the file didn't matter at this point since the server was going to overwrite it — what mattered was the filename and the commit's author field, which now read `root <root@picoc.tf>` thanks to Step 2.

Step 4

Push and let the server-side hook do the rest

Running git push sent the commit to the remote. The server has a hook that inspects each incoming commit's author metadata; when it saw a commit for flag.txt authored by root:root@picoctf, it matched the rule in the README and rewrote flag.txt server-side with the real flag before accepting the push. The push output confirmed it directly: "Author matched and flag.txt found in commit... Congratulations! You have successfully impersonated the root user" followed by the flag itself.

Step 5

Retrieve the flag

Since the flag was printed straight to the terminal in the push output, there was no extra extraction step — I just copied picoCTF{1mp3rs0n4t4_g17_345y_367122f4} and submitted it on the challenge page, which confirmed "Correct flag!"

```
Only flag.txt pushed by ``root:root@picoctf`` will be updated with the flag.

GOOD LUCK!
PascuaDiana-academy@webshell:~/challenge$ git config user.name "root"
PascuaDiana-academy@webshell:~/challenge$ git config user.email "root@picoctf"
PascuaDiana-academy@webshell:~/challenge$ echo "placeholder" > flag.txt
PascuaDiana-academy@webshell:~/challenge$ git add flag.txt
PascuaDiana-academy@webshell:~/challenge$ git commit -m "add flag.txt"
[master 04f1c7b] add flag.txt
1 file changed, 1 insertion(+)
create mode 100644 flag.txt
PascuaDiana-academy@webshell:~/challenge$ git push
git@foggy-cliff.picoctf.net's password:
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 277 bytes | 277.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote: Author matched and flag.txt found in commit...
remote: Congratulations! You have successfully impersonated the root user
remote: Here's your flag: picoCTF{1mp3rs0n4t4_g17_345y_367122f4}
To ssh://foggy-cliff.picoctf.net:54145/git/challenge.git
```

MY GIT

General Skills Easy

by Darkraic9492 picoCTF 2026

INSTANCE 1

Launch Instance

Additional details appear once you launch your instance.

HINTS 1

1

How do you specify your Git username and email?

FLAG 1

picoCTF{1mp3rs0n4t4_g17_345y_36

Submit

Correct flag!

5. Flag

`picoCTF{1mp3rs0n4t4_g17_345y_367122f4}`

6. Key Concept / What I Learned

The underlying lesson here is that Git commit authorship (user.name and user.email) is just client-supplied metadata — it's a courtesy field, not an authentication mechanism. Anything that trusts a Git commit's author field as proof of identity, the way this challenge's server-side hook did, can be trivially spoofed by anyone who can push to the repo. It's a good reminder that authorship and authentication are two different things, and that server-side automation needs real verification (signed commits, access controls, etc.) if it's going to make trust decisions based on "who" did something.

7. References

README.md
